

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04

Análisis de Rendimiento Paralelo aplicado al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: Unidad 4 – Cómputo Paralelo y Distribuido

Estudiante: Jeremy Ruperto Gaibor Rodríguez

Equipo PE-U4: Equipo C

PFC de referencia: Tienda Tech

Transformación foco: T1 – Filtrado y selección

Período académico: 2026–2027 PPA

Docente: Gleiston C. Guerrero-Ulloa, M.Sc.

Repositorio individual:

<https://github.com/JeremyGaibor/ta-ind-04-gaibor>

Fecha: Agosto de 2026

1. Introducción y contexto

El procesamiento paralelo y distribuido permite repartir una carga de trabajo entre varias unidades de cómputo. Sin embargo, aumentar la cantidad de recursos no garantiza una reducción proporcional del tiempo de ejecución. En Apache Spark, la cantidad de executors constituye un parámetro relevante y una asignación inadecuada puede producir un uso poco eficiente de los recursos o un mayor tiempo de ejecución [1]. Asimismo, el ajuste de los parámetros de configuración de Spark constituye un problema de optimización que depende de las características de cada aplicación [2].

El presente informe analiza de manera individual los resultados obtenidos en la práctica PE-U4. En el experimento se implementaron cinco transformaciones equivalentes mediante pandas y PySpark sobre un conjunto sintético de 500.000 solicitudes y 14 columnas. Para cada configuración se realizaron cinco repeticiones y se utilizó la mediana como tiempo representativo. Antes de las repeticiones registradas se descartó una ejecución inicial de calentamiento.

La transformación seleccionada como foco individual es T1, correspondiente al filtrado y selección. Esta operación procesa los 500.000 registros de entrada y produce 33.117 registros como resultado.

Los datos utilizados son trazables al repositorio PE-U4 del Equipo C:

<https://github.com/ffarinangog2/pe-u4-spark-equipo-c>

Las mediciones adicionales de T1 con uno y dos executors se incorporaron en el commit 6b363ab. Los resultados se encuentran registrados en `resultados/tiempos_crudos.csv` y `resultados/tiempos_resumen.csv`.

El entorno documentado en el proyecto utiliza PySpark 3.5.0, Python 3.12.10, OpenJDK 17.0.20 y Windows 11. Spark fue ejecutado en modo Standalone con el master `spark://127.0.0.1:7077`. Cada executor fue configurado con un core y 512 MiB de memoria. Además, el número de particiones, el paralelismo por defecto y el máximo de cores se ajustaron al número de executors utilizado en cada configuración.

2. Análisis propio de los resultados de speedup

La comparación general entre pandas y PySpark utiliza el tiempo mediano de pandas y el tiempo obtenido por Spark con cuatro executors. El speedup se determina mediante:

$$S = \frac{T_{\text{pandas}}}{T_{\text{spark}}}$$

La Tabla 1 resume los resultados de las cinco transformaciones.

Cuadro 1: Resultados generales de PE-U4 con cuatro executors.

Transf.	T_{pandas} (s)	T_{spark} (s)	S	E
T1	0.2119435	0.3456639	0.6131	0.1533
T2	0.1171350	0.6874908	0.1704	0.0426
T3	0.2888583	0.8657363	0.3337	0.0834
T4	2.2658587	1.8100053	1.2519	0.3130
T5	1.5252129	1.3564848	1.1244	0.2811

Los resultados muestran que T4 y T5 alcanzaron un speedup superior a uno, mientras que T1, T2 y T3 fueron más rápidas mediante pandas. En el caso de T1 se obtuvo un speedup de 0,613.1, por lo que la ejecución de Spark con cuatro executors necesitó más tiempo que pandas.

Sen et al. muestran que distintas consultas de Spark pueden requerir niveles diferentes de paralelismo y que una asignación inadecuada de executors puede afectar el rendimiento y el uso de recursos [1]. Por su parte, Cheng et al. estudian el ajuste de parámetros de Spark mediante modelos de predicción y optimización del rendimiento [2].

2.1. Escalado de T1

Para estudiar el escalado de T1 se toma como referencia la ejecución Spark con un executor:

$$T_1 = 0,313.484.6 \text{ s.}$$

Los resultados obtenidos fueron los siguientes:

Cuadro 2: Escalado experimental de T1.

N	T_N (s)	$S(N)$	$E(N)$
1	0.3134846	1.0000	1.0000
2	0.8624843	0.3635	0.1817
4	0.3456639	0.9069	0.2267

Para dos executors:

$$S(2) = \frac{T_1}{T_2} \tag{1}$$

$$= \frac{0.3134846}{0.8624843} \tag{2}$$

$$= 0.3635. \tag{3}$$

Para cuatro executors:

$$S(4) = \frac{T_1}{T_4} \tag{4}$$

$$= \frac{0.3134846}{0.3456639} \tag{5}$$

$$= 0.9069. \tag{6}$$

En ninguna de las configuraciones se obtuvo una aceleración respecto de la ejecución con un solo executor. La configuración con dos executors presentó el peor resultado.

Sus cinco tiempos fueron 0,862.484.3 s, 0,375.832.5 s, 1,171.479.9 s, 0,275.453.9 s y 1,490.659.2 s. La diferencia entre el menor y el mayor valor evidencia una variabilidad considerable en esta configuración.

2.2. Contraste con la Ley de Amdahl

La Ley de Amdahl establece el límite del speedup que puede alcanzarse cuando una fracción p de un programa puede ejecutarse en paralelo [3]:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (7)$$

El experimento no midió de forma independiente la fracción paralelizable p . Por ello, se utiliza $p = 1$ únicamente como una referencia ideal superior, es decir, como el caso en el que no existe fracción serial ni sobrecarga. Esta elección no implica afirmar que T1 sea completamente paralelizable durante una ejecución real.

Bajo esta referencia ideal:

$$S_{\text{teo}}(N) = N.$$

Para dos executors, la desviación relativa entre el valor medido y la referencia ideal es:

$$\Delta_2 = \frac{S_{\text{med}} - S_{\text{teo}}}{S_{\text{teo}}} \times 100 \quad (8)$$

$$= \frac{0.3635 - 2}{2} \times 100 \quad (9)$$

$$= -81.83 \%. \quad (10)$$

Para cuatro executors:

$$\Delta_4 = \frac{0.9069 - 4}{4} \times 100 \quad (11)$$

$$= -77.33 \%. \quad (12)$$

Por tanto, los resultados experimentales se encuentran claramente por debajo del límite ideal de Amdahl. El speedup real es menor que uno tanto con dos como con cuatro executors.

Esto no contradice la Ley de Amdahl. La ecuación representa un límite superior bajo el supuesto de que el paralelismo no introduce sobrecargas adicionales. Una ejecución real de Spark incluye planificación, coordinación y distribución de tareas, por lo que puede producir un speedup inferior a uno cuando dichos costos superan el beneficio del trabajo paralelo.

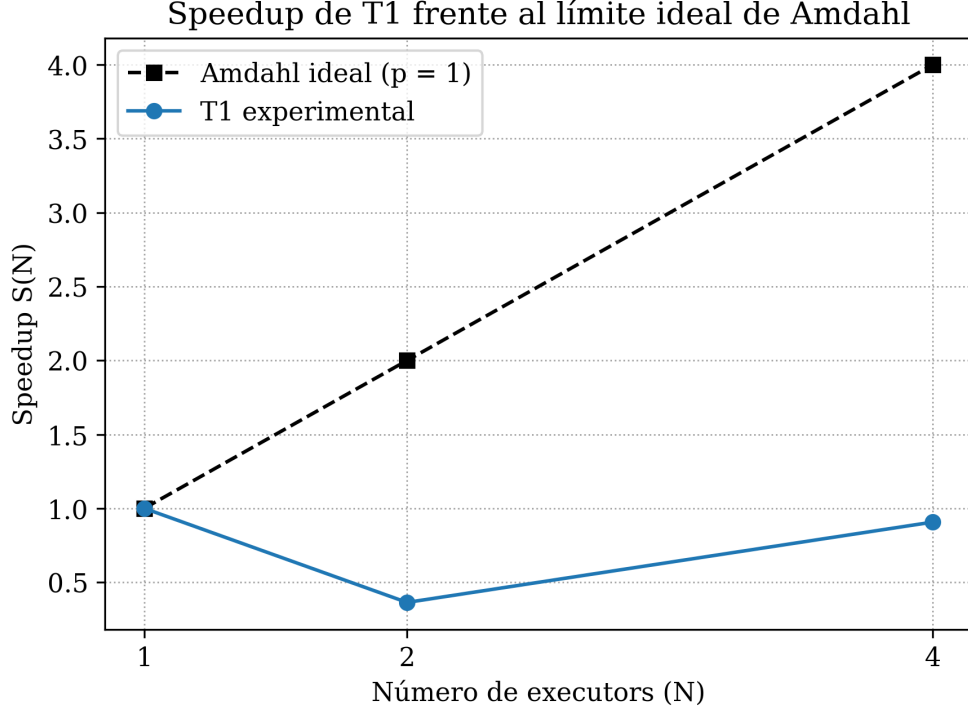


Figura 1: Speedup experimental de T1 frente al límite ideal de Amdahl. Elaboración propia.

La Fig. 1 muestra que el comportamiento experimental se encuentra considerablemente por debajo del límite ideal. Con dos executors se produce una caída marcada del rendimiento y, aunque con cuatro se observa una recuperación, el speedup permanece por debajo de uno.

3. Diagnóstico de la fracción no escalable

La métrica de Karp-Flatt permite estimar experimentalmente la fracción no escalable a partir del speedup observado y del número de unidades de procesamiento [4]:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (13)$$

Para dos executors:

$$e_2 = \frac{\frac{1}{0.3635} - \frac{1}{2}}{1 - \frac{1}{2}} \quad (14)$$

$$\approx 4.5026. \quad (15)$$

Para cuatro executors:

$$e_4 = \frac{\frac{1}{0.9069} - \frac{1}{4}}{1 - \frac{1}{4}} \quad (16)$$

$$\approx 1.1369. \quad (17)$$

Los valores obtenidos son superiores a uno. En estas condiciones no deben interpretarse como una fracción serial física convencional, puesto que el speedup observado es inferior a uno y la ejecución paralela presenta una penalización respecto de la referencia.

Además, e disminuye de 4,502.6 con dos executors a 1,136.9 con cuatro. Por tanto, la pérdida observada no se comporta como una fracción serial aproximadamente constante. La configuración con dos executors presenta una penalización particularmente alta.

3.1. Sobrecarga observada

El primer mecanismo relacionado con los resultados es la planificación y coordinación del trabajo. Spark debe distribuir las tareas entre los recursos disponibles y el nivel de paralelismo seleccionado puede influir en el tiempo total. Sen et al. estudian específicamente la selección del número de executors para consultas Spark SQL y muestran que distintas consultas pueden requerir diferentes niveles de paralelismo [1].

El segundo mecanismo corresponde a la configuración de recursos. En el experimento, `spark.executor.instances`, `spark.cores.max`, `spark.sql.shuffle.partitions` y `spark.default.parallelism` se modifican de acuerdo con el número de executors. Cheng et al. muestran que la configuración de Spark tiene una influencia significativa sobre su rendimiento y plantean su ajuste como un problema de optimización [2].

El tercer elemento es la variabilidad observada en las propias mediciones. Con dos executors se registraron tiempos comprendidos entre aproximadamente 0,275 s y 1,491 s. Esta dispersión muestra que la configuración no presentó un comportamiento estable durante las cinco repeticiones.

La eficiencia del paralelismo se determina mediante:

$$E(N) = \frac{S(N)}{N}. \quad (18)$$

Para dos executors:

$$E(2) = \frac{0.3635}{2} = 0.1817.$$

Para cuatro executors:

$$E(4) = \frac{0.9069}{4} = 0.2267.$$

Por tanto, la eficiencia fue aproximadamente 18,17 % con dos executors y 22,67 % con cuatro. Aunque la configuración de cuatro executors mejora respecto de la de dos, ambas se encuentran lejos del comportamiento ideal.

4. Propuesta de integración al PFC

En el PFC Tienda Tech, el procesamiento distribuido se propone para el análisis histórico de pedidos, ventas y movimientos de inventario. No se plantea su uso dentro del flujo transaccional encargado de registrar directamente una compra, debido a que este proceso requiere una respuesta inmediata y trabaja con una cantidad reducida de información por solicitud.

La integración se realizaría mediante procesamiento por lotes. De forma periódica se extraerían de PostgreSQL los registros históricos necesarios y se entregarían a un proceso Spark. Este proceso podría agrupar las ventas por producto y período, analizar el movimiento del inventario y generar indicadores de demanda.

La salida sería almacenada como información analítica preparada para consulta. El módulo de reportes utilizaría estos resultados para evitar el procesamiento completo del histórico cada vez que un usuario solicite información.

La decisión funcional soportada por esta integración sería la priorización de la reposición del inventario. A partir de los resultados podrían identificarse productos con alta demanda, bajo movimiento o necesidad de reposición.

Se propone una ejecución por lotes debido a que este procesamiento no necesita realizarse durante la compra de un cliente. La tarea podría ejecutarse periódicamente una vez acumulada nueva información.

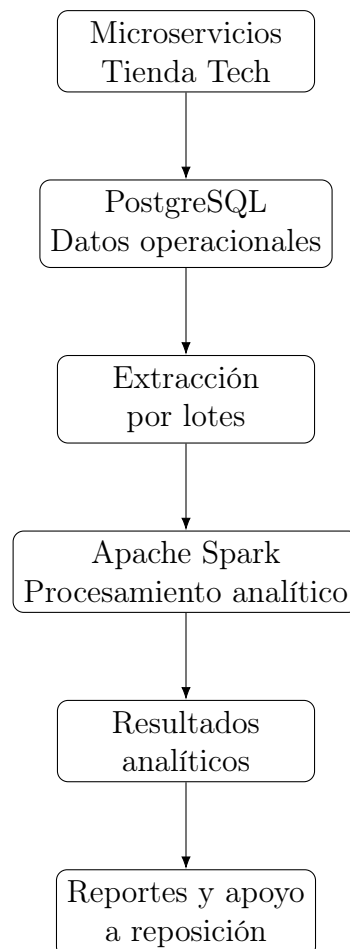


Figura 2: Propuesta de integración del procesamiento distribuido en Tienda Tech. Elaboración propia.

Como se observa en la Fig. 2, Spark se ubica después de la persistencia de los datos operacionales. De esta forma, el procesamiento analítico se mantiene separado de los microservicios responsables de las operaciones transaccionales.

5. Dimensionamiento y viabilidad

El modelo para estudiar la rentabilidad del procesamiento distribuido considera un tiempo secuencial:

$$T_{\text{sec}}(n) = \alpha n,$$

y un tiempo distribuido:

$$T_{\text{dist}}(n) = \beta + \frac{\gamma n}{N}.$$

El umbral de rentabilidad se expresa mediante:

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \alpha > \frac{\gamma}{N}. \quad (19)$$

Los parámetros α , β y γ deben obtenerse mediante un ajuste sobre mediciones experimentales. Los datos disponibles para T1 corresponden a un único volumen de 500.000 registros. Por esta razón, no es posible identificar de forma independiente los tres parámetros del modelo utilizando solamente la evidencia actual.

En consecuencia, no se presenta un valor numérico definitivo para n^* . Utilizar las configuraciones de uno, dos y cuatro executors como si representaran distintos valores de n no sería correcto, ya que en esas pruebas cambia N , pero el volumen de entrada permanece constante.

Para determinar el umbral sería necesario repetir T1 con varios tamaños de entrada y mantener controladas las demás condiciones del experimento. Gulino et al. estudian modelos para predecir el tiempo de ejecución de aplicaciones Spark considerando características del volumen procesado y los recursos asignados [5]. De manera relacionada, Lattuada et al. estudian la asignación de recursos en aplicaciones Spark mediante modelos orientados a cumplir objetivos de rendimiento [6].

Mientras no se determine experimentalmente dicho umbral, para volúmenes similares al evaluado se mantendría una solución basada en pandas o consultas SQL, debido a que T1 no presentó una ventaja de rendimiento mediante Spark.

Para una futura implementación, el procesamiento distribuido se ejecutaría como una tarea independiente por lotes. Entre los principales riesgos se encuentran la variabilidad del rendimiento, el costo de mantener infraestructura adicional, un crecimiento de datos insuficiente para justificar Spark y la sincronización de la información procedente de distintos microservicios.

6. Postura crítica y alternativas

Las mediciones disponibles indican que Spark no es actualmente la opción más eficiente para T1 con 500.000 registros. pandas obtuvo una mediana de 0,211.943.5 s, mientras que Spark con cuatro executors necesitó 0,345.663.9 s. Esto produjo un speedup de 0,613.1.

La evidencia experimental también muestra que aumentar la cantidad de executors no produjo una mejora progresiva. Sen et al. señalan que el nivel adecuado de paralelismo depende de la consulta ejecutada y que la asignación del número de executors influye en el rendimiento y la utilización de recursos [1].

Una primera alternativa es utilizar **pandas**. Para el volumen evaluado, esta opción obtuvo un menor tiempo y evita parte de la coordinación requerida por un motor distribuido.

La segunda alternativa consiste en utilizar **PostgreSQL** para operaciones de filtrado y agregación cuando el volumen pueda procesarse de forma adecuada directamente en la base de datos. Esta opción evita introducir un componente adicional en la arquitectura para cargas que todavía no requieren procesamiento distribuido.

Spark se reservaría para escenarios donde el crecimiento de los históricos de pedidos, ventas e inventario haga insuficiente una solución basada en un solo nodo. Antes de adoptarlo sería necesario realizar pruebas con diferentes volúmenes para comprobar que el paralelismo compensa la sobrecarga introducida.

Por tanto, Spark puede considerarse una alternativa para el componente analítico futuro de Tienda Tech, pero las mediciones actuales no justifican utilizarlo para T1 bajo las condiciones evaluadas.

7. Conclusiones

La transformación T1 no obtuvo una aceleración al incrementar el número de executors. Tomando como referencia Spark con un executor, el speedup fue 0,363.5 con dos executors y 0,906.9 con cuatro. Ambos valores fueron inferiores a uno, por lo que agregar recursos no redujo el tiempo respecto de la configuración base.

La métrica de Karp-Flatt produjo valores de 4,502.6 para dos executors y 1,136.9 para cuatro. Estos resultados, junto con la dispersión de los tiempos observada con dos executors, indican que la ejecución estuvo fuertemente afectada por costos adicionales y que la pérdida de rendimiento no puede explicarse únicamente mediante una fracción serial constante.

Finalmente, Spark tendría mayor sentido en Tienda Tech como componente de procesamiento por lotes para históricos de pedidos, ventas e inventario que como parte del flujo transaccional. Sin embargo, antes de adoptarlo debe determinarse experimentalmente el volumen a partir del cual la distribución resulta rentable mediante nuevas mediciones con distintos tamaños de entrada.

8. Declaración de uso de inteligencia artificial generativa

Se utilizó ChatGPT, de OpenAI, como herramienta de apoyo durante la elaboración de este informe. Su uso estuvo orientado a la revisión de la redacción técnica, organización del contenido según la rúbrica, comprobación de cálculos matemáticos y apoyo en la estructuración del documento en L^AT_EX.

La herramienta fue utilizada principalmente en las secciones de análisis de speedup, contraste con la Ley de Amdahl, diagnóstico mediante Karp-Flatt, propuesta de integración al PFC, dimensionamiento y revisión de conclusiones.

Los datos experimentales no fueron generados ni modificados mediante inteligencia artificial. Los tiempos analizados proceden de las ejecuciones registradas en el repositorio PE-U4 del Equipo C, incluidas las mediciones adicionales de T1 con uno y dos executors.

La interpretación final de los resultados, la selección de la transformación T1 y las decisiones relacionadas con la propuesta para Tienda Tech fueron revisadas y asumidas por el autor del informe.

Referencias

- [1] R. Sen et al., «AutoExecutor: Predictive Parallelism for Spark SQL Queries,» *Proceedings of the VLDB Endowment*, vol. 14, n.º 12, págs. 2855-2858, 2021. DOI: 10.14778/3476311.3476362.
- [2] G. Cheng, S. Ying y B. Wang, «Tuning Configuration of Apache Spark on Public Clouds by Combining Multi-Objective Optimization and Performance Prediction Model,» *Journal of Systems and Software*, vol. 180, pág. 111028, 2021. DOI: 10.1016/j.jss.2021.111028.
- [3] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.
- [4] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [5] A. Gulino, A. Canakoglu, S. Ceri y D. Ardagna, «Performance Prediction for Data-Driven Workflows on Apache Spark,» en *2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, IEEE, 2020, págs. 1-8. DOI: 10.1109/MASCOTS50786.2020.9285944.
- [6] M. Lattuada, E. Barbierato, E. Gianniti y D. Ardagna, «Optimal Resource Allocation of Cloud-Based Spark Applications,» *IEEE Transactions on Cloud Computing*, vol. 10, n.º 2, págs. 1301-1316, 2022, Early access 2020. DOI: 10.1109/TCC.2020.2985682.